

# ggvmap: honest Voronoi treemaps for ggplot2, in pure R

Loukas Theodosiou

loukesio@gmail.com

**Abstract** A treemap shows how a whole splits into parts by giving each part an area in proportion to its size. A Voronoi treemap does this with irregular, many-sided cells inside any convex outline, such as a circle. The cells are found by trial and correction, so the areas are never exact, only close. `ggvmap` is an R package that computes these maps in plain R, with `ggplot2` as its only required dependency, and draws them as ordinary `ggplot2` plots. Every map reports how close its areas came to the intended values, for the map as a whole (converged) and for each cell (`vm_fit()`). The layout combines the established step rules of the `d3-voronoi-map` library with a second phase, based on Newton’s method, that rescues maps those rules cannot fit. In a reproducible study of 519 maps, every map met the default 1% target; without the second phase, 106 of them would have failed. Next to the two existing R implementations, `ggvmap` is the easiest to install, the fastest for small maps, and the only one that reaches its accuracy target at default settings. On large maps it is much slower than the compiled implementation.

## 1 What this package is for

Many questions in science and everyday life are about how a whole splits into parts: how a country spends its budget, which genes make up a genome, which countries hold the world’s fresh water. A pie chart answers this with slices. A *treemap* answers it with a picture made of tiles, where each tile’s area is its share of the whole. If Brazil holds 13% of the world’s fresh water, its tile covers 13% of the picture.

The classic treemap uses rectangles. A *Voronoi treemap* [1] uses irregular tiles with many sides instead, and fits them inside any convex outline: a circle, a hexagon or a triangle (a shape is *convex* when the straight line between any two of its points stays inside it). These maps are popular in newspaper graphics because they look organic and fit any outline. Figure 1 shows one with ten countries.

Two R packages already draw Voronoi treemaps: **`voronoiTreemap`** [2], which runs its calculation in a web browser, and **`WeightedTreemaps`** [3], which runs it in compiled C++ code and needs the CGAL geometry library to be built during installation. Neither produces a `ggplot2` plot, the format most R users build figures with [4]. **`ggvmap`** fills that gap: the calculation is written in plain R, `ggplot2` is its only required dependency, and the result is an ordinary `ggplot2` object that the rest of the `ggplot2` ecosystem can change.

The rest of this paper explains how the map is built (Section 2), how close it comes to the intended areas and how to check that for any map (Section 3), what the package can draw (Section 4), how it compares with the other two packages (Section 5), and where it falls short (Section 6). The precise mathematics is in Section 7.

## 2 How the map is built

**Pins and pull.** Picture a cork board in the shape of a circle with one pin per country. Every spot on the board belongs to one pin. In an ordinary Voronoi diagram, a spot belongs to the nearest pin. Here, each pin also has a *pull*: a pin with a strong pull claims spots that are a bit farther away. The exact rule is that a spot belongs to the pin with the smallest *squared distance minus pull*. Mathematicians call the resulting picture a *power diagram* [5]. Its useful property is that the border between two neighbouring pins is always a straight line, so every tile is a convex polygon, and the tiles fit together with no gaps and no overlaps.

**Adjusting until the areas are right.** At first the pins are spread out at random and all pulls are similar, so the tiles have the wrong sizes (Figure 1 A). The package then repeats two corrections, one round at a time [6]:

1. *Move each pin to the middle of its tile.* This keeps tiles round and compact rather than long and thin.

2. *Change each pin's pull.* If a tile is smaller than it should be, its pull goes up a little; if it is larger, its pull goes down a little.

After each round, the package adds up every tile's mistake (how much too big or too small it is) and divides by the area of the whole map. When that total falls below 1% (the default), it stops. If it has not reached 1% after 200 rounds (the default limit), it stops anyway and says so. Figure 1 shows the mistakes shrinking from 12.6% after the first round to well under 1% at the end.

**Careful steps.** Taking big correction steps makes the tiles overshoot and wobble back and forth. `ggvmap` therefore uses the cautious step rules of Franck Lebeau's JavaScript library *d3-voronoi-map* [7], translated into R: a pull never changes by more than 10% per round, steps shrink when the total mistake has gone up in recent rounds, and a pin whose pull becomes so strong that it would swallow a close neighbour is corrected. These rules are practical safeguards, not mathematical guarantees, and they do get stuck: most often when one value is much larger than all the others, or when the values range from tiny to huge.

**When the rounds get stuck.** If 25 rounds pass without improvement, the package stops moving the pins and works out the pulls directly. This rests on a mathematical result: for pins that stay where they are, there is always a set of pulls that gives every tile exactly its intended area [8]. Newton's method, the classic technique of improving a guess by following the slope, finds those pulls reliably if it starts with all pulls equal and takes careful steps [9]; in practice it needs only a handful of steps. Maps that the rounds fit on their own are not affected.

**Groups.** When values belong to groups (countries in regions, say), the package first splits the outline into one tile per group, then splits each group tile among its members with the same method. Each of the two steps gets half of the 1% allowance, so the finished map as a whole stays within 1% (Section 7 explains why).

**Very small values.** A tile for a value hundreds of times smaller than the largest one would be too small to see or to label. By default, any value below 1% of the largest is drawn as if it were exactly 1% of the largest (`min_weight_ratio`). Such tiles are drawn *bigger than their value*. `vm_fit()` shows which tiles this affected, and `min_weight_ratio = 0` turns it off.

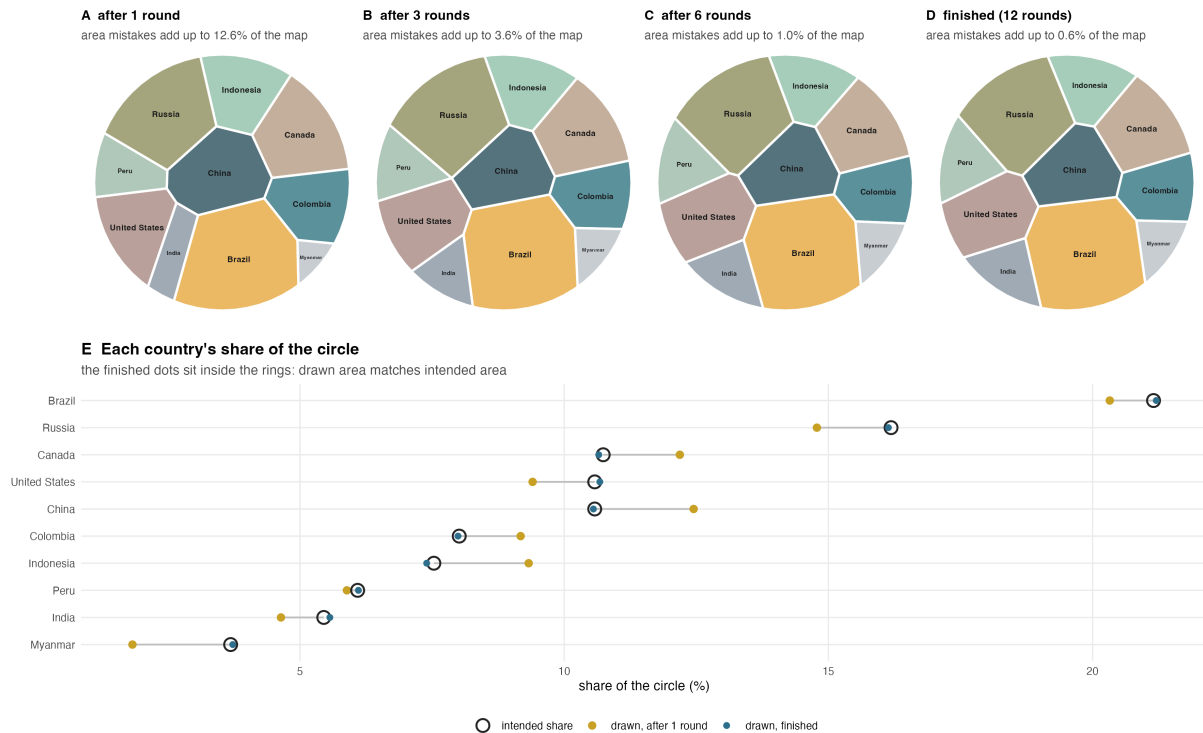


Figure 1: How a map is built. The ten countries with the most renewable fresh water [10], laid out in a circle from the same random start and stopped after 1, 3 and 6 rounds (A–C) and when finished (D). The subtitle of each panel gives the area mistakes of all tiles added up, as a share of the circle; the package stops once this falls below 1%. Pins move toward the middle of their tiles and pulls are adjusted every round, so the tiles shift and grow or shrink. (E) For each country (one row), the open ring is the share of the circle its tile should cover, the yellow dot is the share it covered after round 1, and the blue dot the share in the finished map. After round 1 Myanmar’s tile is far too small and China’s too large; in the finished map every blue dot sits inside its ring.

### 3 Is the picture honest?

A treemap makes one promise: *area equals amount*. Two different questions decide whether a particular map keeps it.

**Are the tiles a proper map?** Yes, always, because of how they are built: each tile is cut out of the outline with straight lines, so tiles cannot overlap, cannot leave gaps, and are always convex. This needs no experiment. The package’s tests still check it on every build (`tests/testthat/test-correctness.R`), the same way one would check that a sorting function returns all of its input: as a guard against programming mistakes, not as a result.

**Does each tile have the right area?** Only approximately, and this is the question that matters. The rounds stop at a 1% total mistake, or earlier if they run out of rounds. So every map carries its own report. Printing the map shows the total mistake and whether the target was met:

```
vm <- voronoi_map(freshwater$share, labels = freshwater$country,
                  group = freshwater$region, clip = clip_circle(), seed = 5)
vm
```

```
Voronoi Map
hierarchical | 6 groups | 30 cells
30 cells | 26 iterations | convergence: 0.789%
Converged: TRUE
```

and `vm_fit()` lists each tile’s intended share next to the share it was drawn with:

```
fit <- vm_fit(vm)
head(fit[order(-abs(fit$rel_error)), c("label", "value_share",
  "target_share", "actual_share", "rel_error")], 3)
```

```
      label value_share target_share actual_share rel_error
25 Middle East 0.009980040 0.009980040 0.012405966 0.24307782
30    Bolivia 0.006986028 0.006986028 0.006519687 -0.06675337
27 New Zealand 0.007984032 0.007984032 0.007699110 -0.03568648
```

The second listing shows the most important lesson about these maps. The map as a whole is within its 1% allowance, yet single small tiles can be off by a noticeable *fraction of their own size*: a tile meant to cover 1% of the map that covers 1.1% is 10% too big. A total measure barely notices this, because the extra area is tiny compared with the whole. For readers this is usually harmless (no one can see a tenth of a percent of a circle), but anyone who reads exact values off small tiles should check `vm_fit()` first, or tighten `convergence_ratio`.

**How close do maps get in practice?** We laid out 519 maps (Table 1): flat maps of 5 to 40 tiles in four outlines, flat maps of 100 tiles, and grouped maps of 30 tiles in 5 groups, each with three kinds of values: *even* (all within a factor of two of each other), *typical* (a few large and many small values) and *extreme* (values spanning several orders of magnitude). Ten different random datasets were drawn for each setting (three at 100 tiles). The script is `data-raw/fit_study.R` in the package repository.

Table 1: Accuracy study. *Converged*: maps that met the 1% target within the default 200 rounds. *Median tile error*: for each map, the middle value of  $|\text{drawn} - \text{intended}| / \text{intended}$  over its tiles; the table gives the middle value over maps. *Worst tile*: the largest such error in any map of the row. Tiles drawn at the `min_weight_ratio` floor are measured against their floored target.

| Maps                          | Values  | Layouts | Converged  | Median rounds | Median tile error | Worst tile |
|-------------------------------|---------|---------|------------|---------------|-------------------|------------|
| Flat, 5-40 tiles              | even    | 160     | 160 of 160 | 14.0          | 0.67%             | 20%        |
|                               | typical | 160     | 160 of 160 | 31.0          | 0.76%             | 214%       |
|                               | extreme | 160     | 160 of 160 | 44.0          | 0.84%             | 115%       |
| Flat, 100 tiles               | even    | 3       | 3 of 3     | 35.0          | 0.71%             | 4%         |
|                               | typical | 3       | 3 of 3     | 78.0          | 0.89%             | 40%        |
|                               | extreme | 3       | 3 of 3     | 49.0          | 0.04%             | 10%        |
| Grouped, 30 tiles in 5 groups | even    | 10      | 10 of 10   | 1.0           | 0.35%             | 2%         |
|                               | typical | 10      | 10 of 10   | 5.5           | 0.64%             | 35%        |
|                               | extreme | 10      | 10 of 10   | 9.5           | 1.39%             | 30%        |

Three things stand out. First, the 1% target was met in 519 of 519 maps (Table 1, *Converged*). Without the second phase described in Section 2, 106 of these maps would have got stuck (77 with extreme, 26 with typical and 3 with even values), some with area mistakes adding up to 51% of the map. Second, the typical tile is drawn very close to its intended size: the median tile error was 0.71% over all maps, and below 3% in every setting. Third, single small tiles can still be well off. The worst tile of the study was drawn at 3.1 times its intended size, in a map that as a whole was within its 1% allowance (Table 1, *Worst tile*). In every map the tiles covered the outline exactly (largest difference  $1e-15$ , which is rounding error in the computer's arithmetic).

**Same values, different picture.** The starting positions of the pins are random. Different seed values therefore give different arrangements of the same data, all equally valid. The arrangement carries no meaning: neighbouring tiles are not more similar than distant ones, and the position of a tile says nothing about its value. Only areas (and the colours the user chooses) carry information. Setting seed makes a figure reproducible.

## 4 What the package can draw

**Three steps.** `voronoi_map()` computes the map, `ggvmap()` draws it, and the `vm_add_*()` functions add decorations: an outer ring naming the groups (`vm_add_ring()`), extra text such as values (`vm_add_labels()`), flags (`vm_add_flags()`) and pictures (`vm_add_images()`). Each step takes the result of the previous one, so they chain with R's pipe `|>` (Figure 2 A–C). Keeping the computation separate from the drawing is deliberate: the computation is the slow and random part, so it is done once, checked (Section 3), and the same checked map is then used for every figure made from it.

**Changing the look.** Appearance can be set for the whole map, for one group, or for one tile (Figure 2 D). Where an option takes one value for everything, it also takes a list of values named after tiles or groups, so making one country bold, or outlining one region, takes one argument. Because the result is a `ggplot2` plot, titles, themes and fonts work as for any other `ggplot2` figure.

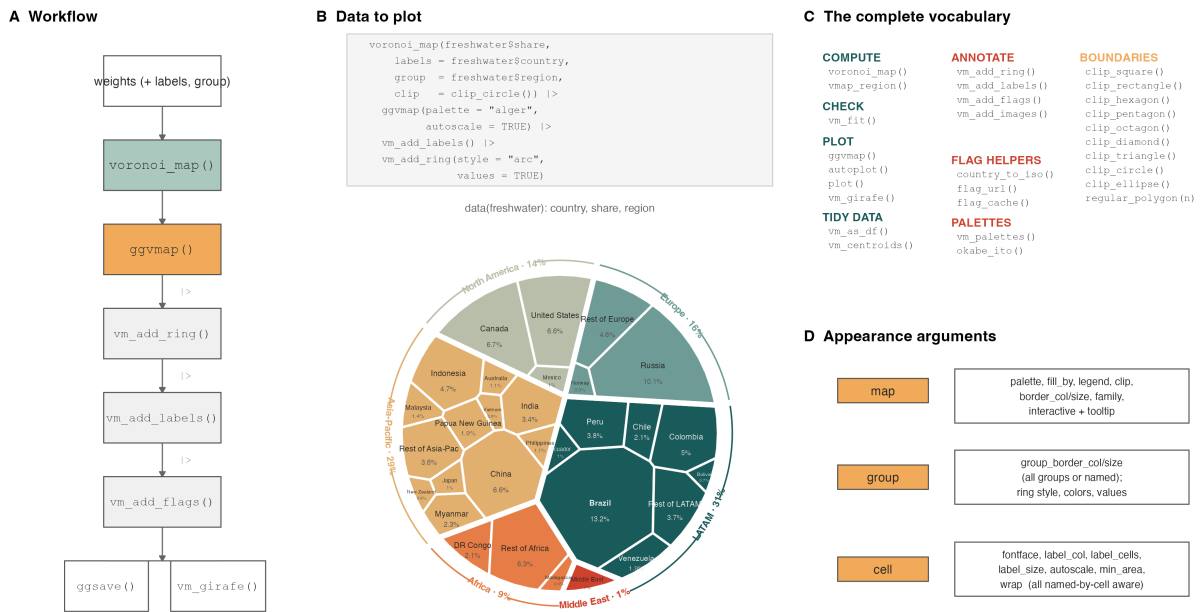


Figure 2: The design of `ggvmap`. (A) The steps: values (with optional names and groups) go into `voronoi_map()`, which computes the map; `ggvmap()` draws it; decorations are added with the pipe; the result is saved with `ggsave()` or made interactive with `vm_girafe()`. (B) The nine lines of code shown produce the finished figure below them from the package's `freshwater` data (each country's share of the world's renewable fresh water, 2022 [10]). Each tile is a country, its area is the country's share, its colour and slice of the circle show the region, and the outer ring gives each region's total. (C) Every function the package offers, grouped by purpose; `vm_fit()` is the per-tile check of Section 3, and `vmap_region()` fits maps inside non-convex outlines such as city boundaries (it needs the optional `polyclip` package and is not covered in this paper). (D) Options that change the look, grouped by whether they act on the whole map, one group or one tile.

Figure 3 shows the range of maps from one dataset: nine outlines, grouped maps, the two ring styles, colour by value, and flags. Figure 4 shows four examples from different fields. Tiles too small to hold their name get smaller text (`autoscale`) or no text (`min_area`), so maps with many small values stay readable without manual adjustment.

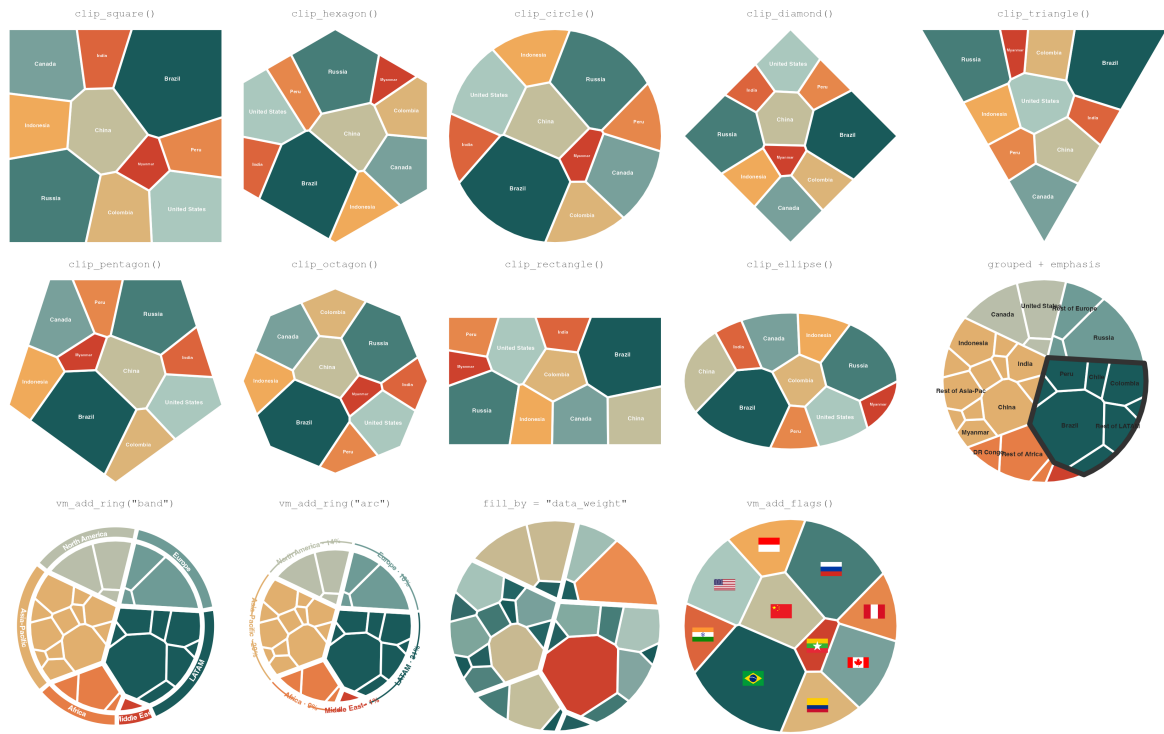


Figure 3: One dataset, fourteen maps. Rows 1–2: the same ten values in each of the nine built-in outlines; the outline is a free design choice, and in each the tile areas match the values. Row 3, left to right: all 30 countries grouped by region, with Latin America outlined in black; a ring of solid bands naming the regions; a thin ring with each region's total share; colour showing each country's value, so large values stand out twice, by area and by colour; and national flags.

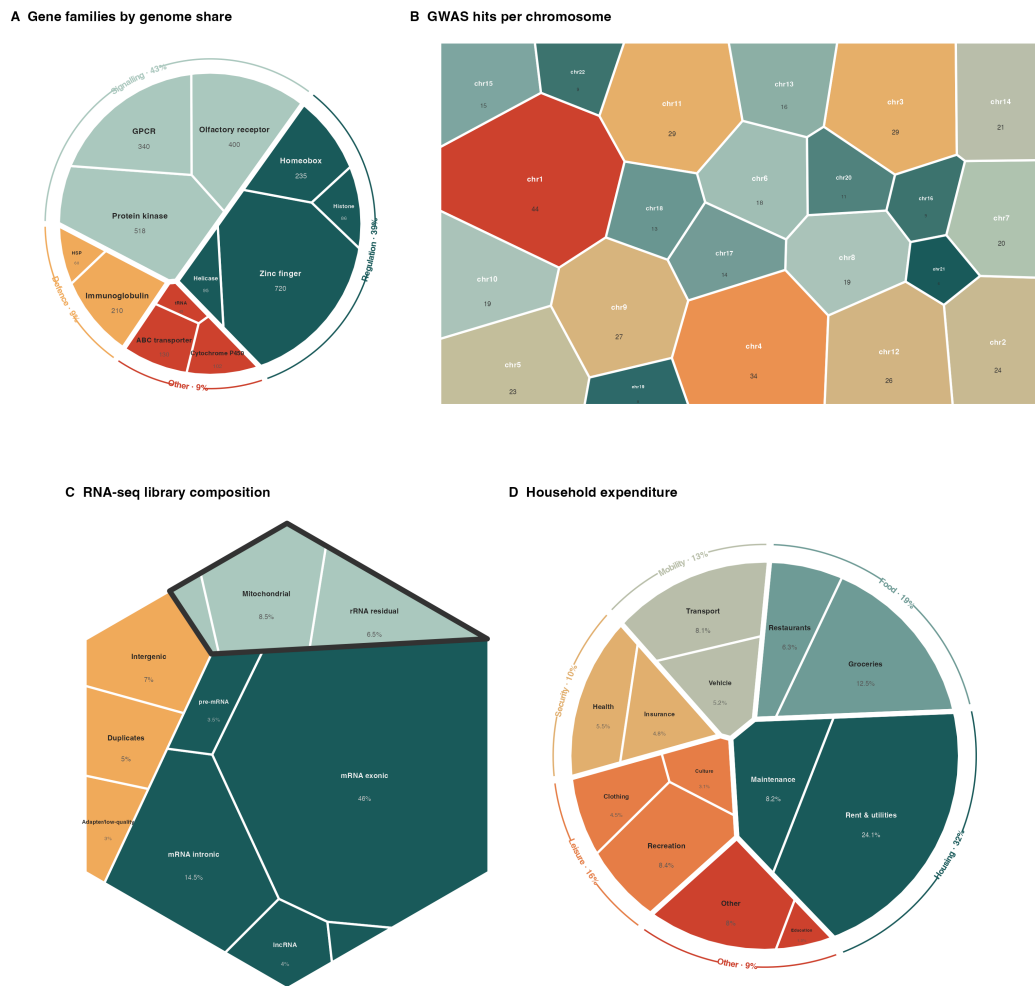


Figure 4: Example uses, with made-up but realistic numbers; each panel's full script is in `paper/figures/fig3_usecases.R` and can be reused with real data. (A) Twelve large gene families, sized by number of genes and grouped by function. (B) Genetic association hits per human chromosome, in a rectangle, coloured by the same value as the area. (C) Where the reads of a sequencing experiment come from, with contamination outlined and the smallest classes left unlabelled. (D) Household spending, grouped by need.

## 5 How it compares

Figure 5 compares `ggvmap` with the two other R packages, each at its default settings, laying out the same values in the same circle. **Installing** (Figure 5 A): `ggvmap` needs 17 other packages and no compiling; `WeightedTreemaps` needs 36 plus compiling `CGAL`; `voronoiTreemap` needs a browser environment. **Code** (Figure 5 B): the same grouped map takes about as much code in `ggvmap` as in `WeightedTreemaps`, and less than in `voronoiTreemap`. **Speed** (Figure 5 C): `ggvmap` is faster up to 50 tiles (151 ms versus 737 ms at 10 tiles), because the compiled package has a fixed start-up cost. It grows slower quickly with size: at 500 tiles it took 4.5 min against 23.2 s. `Treemaps` usually show tens of categories, where the difference is small; for many hundreds of tiles, `WeightedTreemaps` is the better tool. **Accuracy** (Figure 5 D): by the total-mistake measure of Section 2, `ggvmap` stayed within its 1% target in 26 of 26 layouts; `WeightedTreemaps`, at its own default settings, which stop after a fixed amount of work rather than at an accuracy target, ended between 2.6% and 24.5%. Both packages can be made more accurate by allowing more work, so panel D compares defaults, not the best each can do. `voronoiTreemap` runs inside the browser, so its speed and accuracy cannot be measured from R.

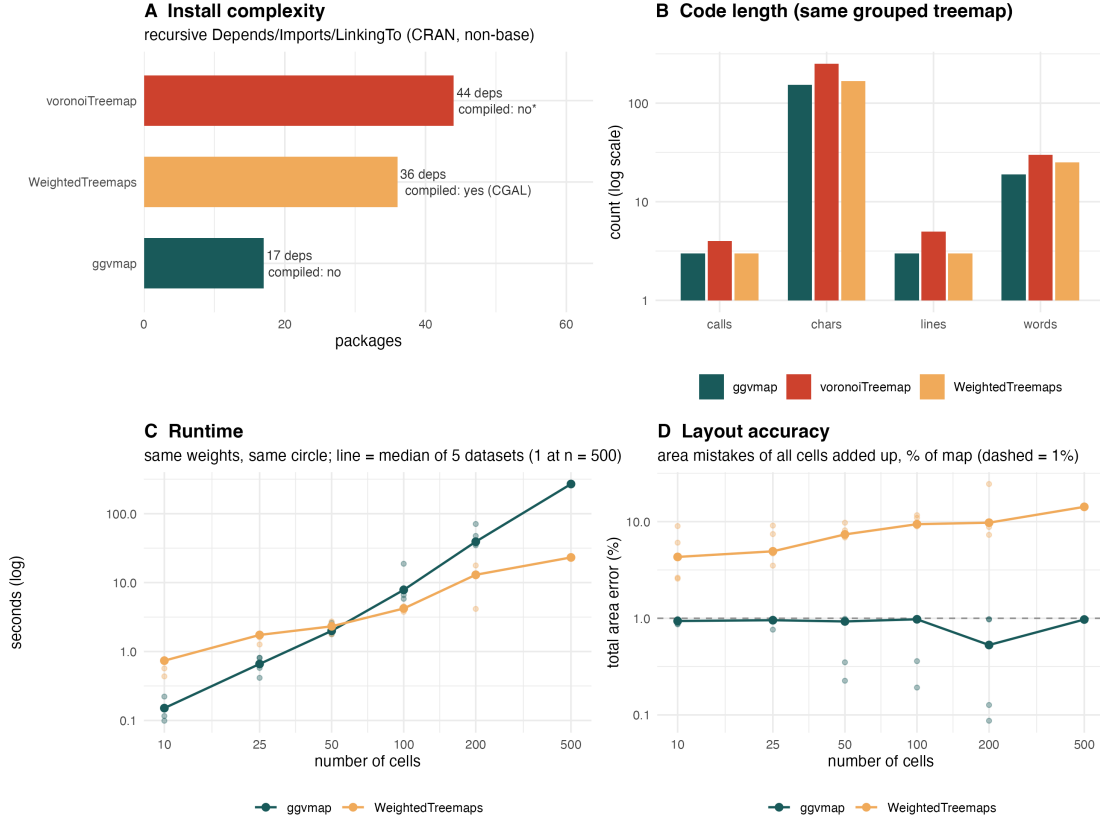


Figure 5: Comparison with the two other R implementations, each at its default settings (protocol in Section 7; script paper/figures/fig4\_benchmark.R). (A) Other packages that must be installed first; ggmap needs only ggplot2 and the packages ggplot2 itself needs. (B) Amount of code for the same grouped map (log scale; less is better). (C) Time to compute one layout of random values in a circle, both axes on log scales; faint dots are single datasets, lines connect the medians. (D) Total area mistake of the finished layout, as in Figure 1; the dashed line is ggmap’s 1% target. voronoiTreemap appears only in A–B because it computes in the browser.

## 6 Limitations

- **Speed on large maps.** For every tile, ggmap cuts the outline with one straight line per other tile, so the work per round grows with the square of the number of tiles. Faster methods are well known: a power diagram can be computed from a three-dimensional convex hull in time proportional to  $n \log n$  [5], which is what compiled geometry libraries do. We chose the simple method because it fits in a few dozen lines of plain R and is fast enough for tens of tiles; the price is shown in Figure 5 C.
- **No overall guarantee.** The second phase rests on a theorem, but it works with whatever pin positions the first phase left, and both phases share one budget of rounds (`max_iter`). The package therefore always measures the result and says when the target was missed.
- **Small tiles.** A map within 1% overall can still have small tiles that are off by a noticeable fraction of their own size, and by default tiny values are drawn at a minimum size (Section 2). Both are visible in `vm_fit()`.
- **No labels outside the map.** Tiles too small for text lose their label; lines leading to labels outside the map are not yet available.

## 7 Methods

**Power diagram.** Each tile  $i$  has a site  $s_i \in \mathbb{R}^2$  and a weight  $w_i$  (the “pull”);  $|x|$  denotes Euclidean length. The power distance from a point  $p$  to site  $i$  is  $|p - s_i|^2 - w_i$ , and tile  $i$  is the set of points of the outline  $P$  whose power distance to  $i$  is smallest. Setting the power distances to  $i$  and  $j$  equal gives  $2(s_j - s_i) \cdot p = |s_j|^2 - |s_i|^2 - w_j + w_i$ , a straight line, so each tile is the intersection of  $P$  with  $n - 1$  half-planes. ggmap computes it by clipping  $P$  against each half-plane in turn (Sutherland–Hodgman clipping),  $O(n^2)$  work per diagram. This is a power diagram, not an additively weighted Voronoi diagram, which uses  $|p - s_i| - w_i$  and has curved borders [1]. If a tile becomes

empty, a tiny placeholder triangle at its site keeps the bookkeeping intact; its area is far below the target, so such a map cannot be reported as converged.

**Iteration.** Targets are  $A_i = A(P) \tilde{v}_i / \sum_j \tilde{v}_j$ , where  $\tilde{v}_i = \max(v_i, \rho \max_j v_j)$  and  $\rho$  is `min_weight_ratio` (default 0.01). Sites start at well-spread random positions (best-candidate sampling) and weights at  $w_i = A_i / \pi$ . Each round, following [6] with the heuristics of [7]: (i) each site moves toward its tile’s centroid by a factor  $1 - 0.5f$ ; (ii) each weight is multiplied by  $A_i / a_i$ , where  $a_i$  is the current area, clamped to  $[0.9 + 0.1f, 1.1 - 0.1f]$ ; (iii) when a heavier site  $h$  and lighter site  $l$  satisfy  $|s_h - s_l|^2 < w_h - w_l$ , so that  $l$ ’s tile would vanish,  $w_l$  is raised to make the inequality an equality. Here  $f$  is the fraction of the last ten rounds in which the total error increased (0 in the first ten rounds), which damps steps when the layout oscillates. The total error is  $E = \sum_i |a_i - A_i| / A(P)$ ; iteration stops when  $E < \text{convergence\_ratio}$  (default 0.01), after 25 rounds without a new lowest  $E$ , or after `max_iter` – `min(30, floor(max_iter / 4))` rounds (default 200, so 170).

**Second phase.** If the first phase ends without reaching the target, the sites are fixed at the positions of the round with the lowest  $E$  and the weights are solved for directly. For fixed distinct sites, the map  $w \mapsto a(w)$  is the gradient of a concave function (the dual of a semi-discrete optimal-transport problem), and weights with  $a(w) = A$  exist and are unique up to an additive constant [8]. Moving the border between neighbours  $i$  and  $j$  by changing  $w_j$  gives  $\partial a_i / \partial w_j = -L_{ij} / (2|s_i - s_j|)$ , where  $L_{ij}$  is the length of their shared border, and  $\partial a_i / \partial w_i = -\sum_{j \neq i} \partial a_i / \partial w_j$ . Starting from  $w = 0$  (an ordinary Voronoi diagram, in which every tile contains its site and is non-empty), each Newton step solves this system with  $w_1$  fixed and halves the step until every tile keeps at least half of the smallest starting or target area and the residual  $|A - a|_2$  has fallen by a factor  $1 - \tau/2$ , where  $\tau$  is the step length; this damped Newton method converges globally [9]. Phase 2 may use the remaining rounds of `max_iter`. The returned object records  $E$ , the number of rounds of both phases, and `converged = (E < convergence_ratio)`.

**Grouped maps.** Floored values  $\tilde{v}_i$  are summed per group, the outline is first divided among groups, and each group tile is then divided among its members, each level with tolerance `convergence_ratio/2`. Let  $G_g$  and  $\hat{G}_g$  be the target and drawn area of group  $g$ , and  $\hat{A}_i = \hat{G}_g \tilde{v}_i / \sum_{j \in g} \tilde{v}_j$  the member target inside the drawn group tile. By the triangle inequality,  $\sum_i |a_i - A_i| \leq \sum_i |a_i - \hat{A}_i| + \sum_g |\hat{G}_g - G_g|$ , so if both levels are within half the tolerance, the whole map is within the tolerance. `converged` reports the whole-map error; `vm$groups$fit` reports each level.

**Software tests.** `tests/testthat/test-correctness.R` checks, on random inputs, that every sampled point lies in a tile whose site has the smallest power distance, that tiles cover the outline with no gaps or overlaps, that tiles are convex, and that grouped member tiles fill their group tile. These are checks of the implementation, since the properties hold by construction.

**Accuracy study.** `data-raw/fit_study.R` lays out flat maps of  $n \in \{5, 10, 20, 40\}$  in a square, circle, hexagon and triangle; flat maps of  $n = 100$  in a circle; and grouped maps of 30 values in 5 groups in a circle. Values are drawn as `Uniform(1, 2)` (*even*), `log-normal(3, 1)` (*typical*) or `log-normal(3, 2)` (*extreme*), with 10 datasets per setting (3 at  $n = 100$ ), all at default settings. Results are in `vignettes/fit_study.csv`.

**Comparison.** Both packages laid out the same `log-normal(3, 1)` values in a circle at default settings, for  $n = 10, 25, 50, 100, 200$  (five datasets each) and 500 (one dataset), timed once per dataset with `system.time()` on one laptop core. Errors are computed from each package’s own reported tile areas, normalised to shares. Dependency counts are the recursive `Depends`, `Imports` and `LinkingTo` on CRAN, without base packages. Code length counts lines, words, characters and function calls of the shortest code producing the same grouped map. Results: `paper/figures/benchmark_runs.csv` (per layout) and `benchmark_results.csv` (medians).

**Availability.** `ggvmap` is at <https://github.com/loukesio/ggvmap> under the MIT licence; the ported `d3-voronoi-map` heuristics keep their BSD-3 licence notice (`inst/COPYRIGHTS`). Every figure and number in this paper is produced by the scripts in `paper/figures/` and `data-raw/`, run by `paper/render.R`.

## Acknowledgements

This package grew out of the author’s Stack Overflow question “Make a circular Voronoi diagram in R” and owes much to Allan Cameron’s answer there. The iteration heuristics are Franck Lebeau’s, from `d3-voronoi-map`.

## Bibliography

- [1] M. Balzer and O. Deussen, “Voronoi Treemaps”, in *Proceedings of the IEEE Symposium on Information Visualization (InfoVis)*, 2005, pp. 49–56. doi: 10.1109/INFVIS.2005.1532128.
- [2] M. Li, E. Mills, and UOB HDRUK, “voronoiTreemap: Voronoi Treemaps with Added Interactivity by Shiny”. [Online]. Available: <https://cran.r-project.org/package=voronoiTreemap>
- [3] M. Jahn and Y. Leifels, “WeightedTreemaps: Generate and Plot Voronoi or Sunburst Treemaps from Hierarchical Data”. [Online]. Available: <https://cran.r-project.org/package=WeightedTreemaps>
- [4] H. Wickham, *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. [Online]. Available: <https://ggplot2.tidyverse.org/>
- [5] F. Aurenhammer, “Power Diagrams: Properties, Algorithms and Applications”, *SIAM Journal on Computing*, vol. 16, no. 1, pp. 78–96, 1987, doi: 10.1137/0216006.
- [6] A. Nocaj and U. Brandes, “Computing Voronoi Treemaps: Faster, Simpler, and Resolution-independent”, *Computer Graphics Forum*, vol. 31, no. 3, pp. 855–864, 2012, doi: 10.1111/j.1467-8659.2012.03078.x.
- [7] F. Lebeau, “d3-voronoi-map: D3 plugin which computes a map (one-level treemap) based on a Voronoi tessellation”. [Online]. Available: <https://github.com/Kcnarf/d3-voronoi-map>
- [8] F. Aurenhammer, F. Hoffmann, and B. Aronov, “Minkowski-Type Theorems and Least-Squares Clustering”, *Algorithmica*, vol. 20, no. 1, pp. 61–76, 1998, doi: 10.1007/PL00009187.
- [9] J. Kitagawa, Q. Mérigot, and B. Thibert, “Convergence of a Newton Algorithm for Semi-Discrete Optimal Transport”, *Journal of the European Mathematical Society*, vol. 21, no. 9, pp. 2603–2651, 2019, doi: 10.4171/JEMS/889.
- [10] Food and Agriculture Organization of the United Nations, “AQUASTAT: FAO's Global Information System on Water and Agriculture -- Renewable Internal Freshwater Resources”. [Online]. Available: <https://data.worldbank.org/indicator/ER.H2O.INTR.K3>